

Analisis Komparatif Algoritma Dijkstra, Branch and Bound, dan Modified Ant Colony Optimization dalam Penyusunan Setlist Orkestra Berdasarkan Transisi Modulasi dan Kurva Energi

Shalom Kurniawan - 24060124120033

Departemen Informatika, Universitas Diponegoro, Semarang, Indonesia

chloudius@undip.ac.id

Abstract— Penyusunan setlist orkestra merupakan masalah optimasi kombinatorial multi-objektif yang bertujuan mencari urutan penampilan lagu terbaik. Permasalahan ini meminimalkan penalti transisi nada dasar (modulasi) berdasarkan jarak Circle of Fifths, perbedaan tempo (BPM), serta menjaga dinamika energi pertunjukan agar terus meningkat (ascending energy curve). Penelitian ini menganalisis tiga strategi perancangan algoritma: Dijkstra Nearest Neighbor (pendekatan greedy lokal), Branch and Bound berbasis DFS (pendekatan eksak dengan pemangkasan cabang), dan Modified Ant Colony Optimization (pendekatan metaheuristik probabilistik yang dipandu feromon dan visibilitas sadar-energi). Ketiga algoritma diuji menggunakan dataset statis berisi 15 repertoar orkestra klasik populer. Hasil analisis membandingkan performa ketiga metode tersebut berdasarkan total cost fungsi objektif, waktu eksekusi komputasi, dan konsistensi kurva energi setlist yang dihasilkan.

Keywords— setlist optimization, Dijkstra, Branch and Bound, Modified Ant Colony Optimization, Circle of Fifths, energy curve, classical repertoire.

I. PENDAHULUAN

Penyusunan setlist orkestra dapat dimodelkan secara matematis sebagai sebuah variasi dari masalah optimasi kombinatorial dengan batasan multi-objektif. Batasan utama dalam permasalahan ini meliputi transisi modulasi antar-nada dasar yang harus berjalan mulus untuk menghindari lonjakan frekuensi yang tidak harmonis, serta pengaturan kurva energi pertunjukan yang harus mengikuti pola perkembangan intensitas tertentu agar perhatian penonton tetap terjaga. Dalam pemodelan graf, setiap lagu bertindak sebagai simpul (node), dan perpindahan antar-lagu merepresentasikan busur (edge) dengan bobot (cost) tertentu yang dihitung dari selisih tempo (BPM), jarak interval harmoni (nada dasar), dan penalti deviasi energi.

Seiring bertambahnya jumlah repertoar dalam daftar lagu yang dimiliki oleh sebuah organisasi orkestra, pencarian urutan optimal secara manual akan menghadapi kendala ledakan ruang pencarian karena banyaknya permutasi yang mungkin ($N!$). Untuk mengatasi kompleksitas tersebut, penerapan strategi algoritma menjadi instrumen penting demi menghasilkan efisiensi komputasi. Penelitian ini berfokus pada analisis performa dan perbandingan metode antara algoritma berbasis pencarian jalur terpendek (Dijkstra Nearest Neighbor), pendekatan eksak deterministik (Branch and Bound berbasis Depth-First Search), serta pendekatan metaheuristik berbasis koloni (Modified Ant Colony Optimization). Kontribusi utama dari makalah ini adalah menyajikan analisis komparatif mengenai sejauh mana ketiga metode tersebut mampu menghasilkan urutan repertoar yang optimal dengan waktu eksekusi yang efisien.

II. BATASAN MASALAH

Untuk menjaga kejelasan dan fokus analisis agar penelitian tidak melebar ke arah yang tidak relevan, penelitian ini dibatasi oleh beberapa hal sebagai berikut:

- 1. Ukuran Dataset:** Pengujian algoritma dibatasi menggunakan dataset statis yang terdiri dari 15 lagu yang merepresentasikan repertoar orkestra klasik populer.
- 2. Model Representasi Nada Dasar:** Perhitungan transisi modulasi didasarkan pada jarak interval absolut nada dasar mayor dan minor standar pada lingkaran perlima (Circle of Fifths), dengan pemetaan nada dasar minor ke relatif mayornya.
- 3. Kuantifikasi Energi:** Kurva energi pertunjukan dimodelkan menggunakan skala nilai diskrit (1 hingga 10) yang ditentukan di awal berdasarkan atribut tempo (BPM) dan dinamika makro aransemen.

4. Kondisi Transisi: Durasi jeda antar-lagu dianggap konstan (tidak dinamis), sehingga fokus optimasi murni berada pada urutan permutasi lagu.

III. DASAR TEORI

A. Representasi Masalah dalam Teori Graf

Dalam pemodelan matematika, masalah optimasi setlist ini dipetakan ke dalam struktur graf berarah lengkap dan berbobot $G = (V, E)$ [1]. Di sini, V mewakili seluruh lagu dalam dataset sebagai simpul, dan E mewakili setiap kemungkinan transisi dari satu lagu ke lagu berikutnya sebagai busur. Setiap busur (u, v) memiliki bobot transisi $W(u, v)$ yang mewakili 'nilai ketidaknyamanan' atau penalti ketika orkestra berpindah dari lagu u ke lagu v . Tujuan utama kita adalah menemukan jalur Hamiltonian (jalur yang mengunjungi setiap simpul tepat satu kali) yang meminimalkan total bobot perjalanan.

B. Pemetaan Nada Dasar Berdasarkan Circle of Fifths dan Circle of Fourths

Dalam teori musik barat, hubungan keharmonisan antar-nada dasar digambarkan secara geometris melalui lingkaran perlima (Circle of Fifths) dan lingkaran perperempatan (Circle of Fourths) [2], [3]. Dua nada dasar yang letaknya bersebelahan hanya berbeda satu nada dalam tangga nadanya, sehingga transisi di antara keduanya terdengar sangat mulus dan natural bagi pendengar. Untuk menghitung jarak terpendek antar-kunci secara matematis, kita menggunakan sistem modular 12 nada dasar. Jarak geodesik terpendek antara kunci lagu u dan v dirumuskan sebagai berikut:

$$\text{Jarak}(u, v) = \min(|\text{Key}_u - \text{Key}_v|, 12 - |\text{Key}_u - \text{Key}_v|)$$

C. Algoritma Dijkstra

Algoritma Dijkstra pertama kali diperkenalkan oleh Edsger Dijkstra pada tahun 1959 [4] untuk memecahkan masalah lintasan terpendek pada graf berbobot non-negatif. Dalam implementasi optimasi setlist ini, algoritma Dijkstra diadaptasi menjadi pendekatan pencari tetangga terdekat (Nearest Neighbor). Cara kerjanya bersifat greedy lokal, yaitu dari lagu pembuka yang ditentukan, algoritma akan selalu memilih lagu berikutnya yang memiliki cost transisi terkecil di antara lagu-lagu yang belum dikunjungi. Algoritma ini berjalan sangat cepat karena tidak melakukan evaluasi balik atau pencarian mundur sama sekali.

D. Algoritma Branch and Bound

Metode Branch and Bound (B&B) diperkenalkan oleh Land dan Doig pada tahun 1960 [5] untuk memecahkan masalah optimasi kombinatorial secara eksak. Berbeda dengan greedy yang hanya mencari solusi lokal, B&B mencari solusi terbaik secara global dengan cara membangun pohon ruang pencarian (state space tree) secara rekursif menggunakan Depth-First Search (DFS) dengan backtracking. Untuk menghindari pemeriksaan seluruh kemungkinan rute (yang berjumlah $N!$), algoritma ini melakukan pemangkasan cabang (pruning). Setiap kali akumulasi cost rute saat ini (cost_so_far) sudah menyamai atau melebihi cost rute terbaik yang sudah ditemukan sementara ($\text{incumbent/best_cost}$), pencarian di cabang tersebut langsung dihentikan dan dibuang karena tidak akan menghasilkan solusi yang lebih baik.

E. Modified Ant Colony Optimization (ACO)

Algoritma Koloni Semut atau Ant Colony Optimization (ACO) dikembangkan oleh Marco Dorigo dkk. pada tahun 1996 [6] sebagai algoritma metaheuristik probabilistik yang terinspirasi dari cara semut menemukan jalur terpendek ke sumber makanan menggunakan jejak feromon. Dalam penelitian ini, kami menggunakan Modified ACO dengan mengubah fungsi visibilitas heuristik agar bersifat dinamis. Visibilitas tidak hanya dihitung dari cost transisi statis antar-kunci dasar, melainkan dikalikan dengan faktor kecocokan energi (energy match) terhadap target kurva energi di setiap posisi setlist. Dengan cara ini, semut akan secara aktif memilih lagu yang tidak hanya harmonis secara nada, tetapi juga pas dengan dinamika panggung yang diinginkan.

IV. METODOLOGI

Metodologi penelitian diawali dengan pengumpulan data 15 lagu orkestra klasik populer (Tabel I). Setiap lagu dimodelkan sebagai simpul dalam graf lengkap. Kunci nada dasar dipetakan ke dalam angka 0 hingga 11 berdasarkan arah jarum jam lingkaran perlima. Hubungan antar-lagu direpresentasikan dalam matriks ketetanggaan berukuran 15 x 15 yang menyimpan bobot transisi numerik hasil perhitungan fungsi objektif.

TABEL I. DATASET 15 REPERTOAR ORKESTRA KLASIK

ID	Komposer & Judul Repertoar	Nada Dasar	BPM	Energi
0	Erik Satie – Gymnopédie No. 1	D (2)	54	1
1	J.S. Bach – Air on the G String	D (2)	60	2
2	Samuel Barber – Adagio for Strings	Db (7)	60	3
3	Johann Pachelbel – Canon in D	D (2)	80	3
4	Edvard Grieg – Morning Mood (Peer Gynt)	E (4)	66	4
5	L. van Beethoven – Symphony No. 9, 3rd Mvt	Bb (10)	60	4
6	J. Strauss II – The Blue Danube Waltz	D (2)	120	5
7	P.I. Tchaikovsky – Swan Lake Theme	D (2)	80	6
8	W.A. Mozart – Symphony No. 40, 1st Mvt	Bb (10)	110	7
9	Antonio Vivaldi – Spring (The Four Seasons)	E (4)	108	7
10	L. van Beethoven – Symphony No. 5, 1st Mvt	Eb (9)	108	8
11	Richard Wagner – Ride of the Valkyries	B (5)	120	9
12	Edvard Grieg – In the Hall of the Mountain King	D (2)	138	9
13	Gioachino Rossini – William Tell Overture	E (4)	150	10
14	P.I. Tchaikovsky – 1812 Overture	Eb (9)	140	10

B. Formulasi Fungsi Objektif (Cost Function)

Fungsi objektif dirancang untuk mengukur total penalti transisi dari lagu u ke lagu v . Persamaan matematika dirumuskan sebagai berikut:

$$\text{TotalCost}(u, v) = \alpha * \text{Cost_Modulation}(u, v) + \beta * \text{Cost_BPM}(u, v) + \text{Cost_Energy}(u, v)$$

C. Penerapan Strategi Eksplorasi Algoritma

Penerapan ketiga algoritma dilakukan dengan urutan pengerjaan sebagai berikut:

- 1. Dijkstra Nearest Neighbor:** Memilih lagu berikutnya dengan bobot terkecil secara langsung dari posisi saat ini. Berjalan instan untuk memberikan baseline solusi awal.
 - 2. Branch and Bound:** Melakukan pencarian mendalam dengan rekursi. Solusi dari Dijkstra digunakan sebagai batas atas awal (initial upper bound) untuk memotong cabang pencarian secara agresif menggunakan logika $\text{cost_so_far} \geq \text{best_cost}$.
 - 3. Modified ACO:** Algoritma ACO yang digunakan dalam penelitian ini merupakan versi modifikasi dari ACO standar. Modifikasi utama terletak pada fungsi visibilitas heuristik dan mekanisme deposit feromon. Eksperimen ini menggunakan parameter kontrol: jumlah semut (num_ants) = 30, iterasi = 150, $\alpha = 1.0$, $\beta = 2.5$, $\rho = 0.5$, dan $Q = 200$. Berikut adalah rincian konfigurasi dan mekanisme kerjanya:
- **Heuristik Visibilitas Energy-Aware:** Pada ACO standar, fungsi visibilitas heuristik hanya dihitung dari kebalikan cost transisi. Dalam modifikasi ini, visibilitas dikalikan dengan faktor kecocokan energi terhadap target kurva energi di setiap posisi setlist. Rumusnya adalah:

$$\eta_{\text{mod}}(i, j, \text{pos}) = \frac{\text{energy_match}(j, \text{pos})}{\text{ADJ}[i][j] + \epsilon}$$

di mana energy_match mengukur seberapa cocok energi lagu j dengan target energi ascending linear pada posisi sekuensial pos :

$$\text{energy_match}(j, \text{pos}) = \frac{1}{1 + |\text{Energy}_j - E_{\text{target}}(\text{pos})|}$$

- **Target Kurva Energi:** Target energi dihitung secara ascending linear dari energi minimum ke energi maksimum dalam dataset:

$$E_{\text{target}}(\text{pos}) = E_{\text{min}} + (E_{\text{max}} - E_{\text{min}}) \cdot \frac{\text{pos}}{N - 1}$$

D. Perancangan Pseudocode Algoritma

Potongan Kode: Pseudocode 1. Dijkstra Nearest Neighbor

```
Dijkstra_Nearest_Neighbor(start_node):
    path = [start_node], visited = [False]*N, visited[start_node] = True
    Untuk step = 1 s.d N-1:
        current = path[-1], min_cost = inf, next_song = -1
        Untuk j di V:
            Jika visited[j] == False dan ADJ[current][j] < min_cost:
                min_cost = ADJ[current][j], next_song = j
        path.append(next_song), visited[next_song] = True
    Return path, cost
```

Potongan Kode: Pseudocode 2. Branch and Bound DFS

```
Branch_and_Bound_DFS(start_node):
    best_path, best_cost = Dijkstra_Nearest_Neighbor(start_node)
    Backtrack(path, cost_so_far):
        Jika len(path) == N:
            Jika cost_so_far < best_cost:
                best_path, best_cost = path, cost_so_far
            Return
        current = path[-1]
        kandidat = [(ADJ[current][j], j) untuk j di V jika visited[j] == False]
        Urutkan kandidat (cheap-first)
        Untuk setiap (edge_cost, next_node) in kandidat:
            Jika cost_so_far + edge_cost >= best_cost: Lanjutkan (pruning)
            path.append(next_node), visited[next_node] = True
            Backtrack(path, cost_so_far + edge_cost)
            path.pop(), visited[next_node] = False
```

Potongan Kode: Pseudocode 3. Modified ACO

```
Modified_ACO(start_node, num_ants, iterations):
    pheromone = [1.0] * N
    Untuk iter = 1 s.d iterations:
        Untuk setiap ant:
            path = [start_node]
            Untuk step = 1 s.d N-1:
                Pilih chosen dari candidates secara probabilistik berdasarkan (pheromone^alpha)*(eta_mod^beta)
                path.append(chosen)
            Evaporasi feromon & update elitist
    Return best_path, best_cost
```

E. Kode Implementasi Program Algoritma Utama (Python)

Berikut di bawah ini dilampirkan potongan kode program asli dalam bahasa pemrograman Python untuk ketiga algoritma utama:

Potongan Kode: Algoritma Dijkstra Nearest Neighbor (dijkstra.py)

```
import heapq

from dataset import ADJ, N

def dijkstra_nearest_neighbor(start=0):
    visited = [False] * N
    path = [start]
    visited[start] = True
    total_cost = 0.0

    for _ in range(N - 1):
        current = path[-1]

        pq = []
        for j in range(N):
            if not visited[j]:
                heapq.heappush(pq, (ADJ[current][j], j))

        min_cost, next_song = heapq.heappop(pq)
        visited[next_song] = True
        path.append(next_song)
        total_cost += min_cost

    return path, total_cost
```

Potongan Kode: Algoritma Branch and Bound DFS (branch_and_bound.py)

```
from dataset import ADJ, N
from dijkstra import dijkstra_nearest_neighbor
def branch_and_bound(start=0):
    # --- Inisialisasi upper bound dari solusi greedy ---
    init_path, init_cost = dijkstra_nearest_neighbor(start)
    best = {"cost": init_cost, "path": init_path[:]}
    stats = {"explored": 0, "pruned": 0}
    def _backtrack(path, visited, cost_so_far):
        stats["explored"] += 1
        # Base case: semua lagu sudah dijadwalkan
        if len(path) == N:
            if cost_so_far < best["cost"]:
                best["cost"] = cost_so_far
                best["path"] = path[:]
            return
        current = path[-1]
        candidates = sorted(
            [(ADJ[current][j], j) for j in range(N) if not visited[j]]
        )
        for edge_cost, next_node in candidates:
            new_cost = cost_so_far + edge_cost
            # --- BOUNDING: pangkas cabang yang sudah tidak mungkin optimal ---
            if new_cost >= best["cost"]:
                stats["pruned"] += 1
                continue
            visited[next_node] = True
            path.append(next_node)
            _backtrack(path, visited, new_cost)
            path.pop()
            visited[next_node] = False
    visited_init = [False] * N
    visited_init[start] = True
    _backtrack([start], visited_init, 0.0)
    return best["path"], best["cost"], stats["explored"], stats["pruned"]
```

Potongan Kode: Algoritma Modified ACO (modified_aco.py)

```
import random
from dataset import ADJ, N, SONGS

def modified_aco(start=0, num_ants=30, iterations=150,
                 alpha=1.0, beta=2.5, rho=0.5, Q=200):
    # Inisialisasi matriks feromon seragam
    pheromone = [[1.0] * N for _ in range(N)]
    # Target kurva energi: ascending linear dari min ke max
    e_min = min(s["energy"] for s in SONGS)
    e_max = max(s["energy"] for s in SONGS)
    target_energy = [
        e_min + (e_max - e_min) * (pos / (N - 1)) for pos in range(N)
    ]

    best_path = None
    best_cost = float("inf")
    convergence_log = []

    for _ in range(iterations):
        iter_paths = []
        iter_costs = []

        for _ in range(num_ants):
            path = [start]
            visited = [False] * N
            visited[start] = True
            ant_cost = 0.0

            for step in range(1, N):
                current = path[-1]
                candidates = [j for j in range(N) if not visited[j]]

                # Hitung bobot probabilitas untuk setiap kandidat
                weights = []
                for j in candidates:
                    energy_match = 1.0 / (1.0 + abs(SONGS[j]["energy"] - target_energy[step]))
                    eta_mod = energy_match / (ADJ[current][j] + 1e-9)
                    tau = pheromone[current][j]
                    weights.append((tau ** alpha) * (eta_mod ** beta))

                # Seleksi roulette wheel
                total_w = sum(weights)
                r = random.random() * total_w
                cumul = 0.0
                chosen = candidates[-1]
                for idx, w in enumerate(weights):
                    cumul += w
                    if cumul >= r:
                        chosen = candidates[idx]
                        break

                ant_cost += ADJ[current][chosen]
                visited[chosen] = True
                path.append(chosen)

            iter_paths.append(path)
            iter_costs.append(ant_cost)

            if ant_cost < best_cost:
                best_cost = ant_cost
                best_path = path[:]

        # --- Evaporasi feromon ---
        for i in range(N):
            for j in range(N):
                pheromone[i][j] = max(pheromone[i][j] * (1.0 - rho), 0.01)

        # --- Deposit feromon (elitist: hanya semut terbaik per iterasi) ---
        best_idx = iter_costs.index(min(iter_costs))
```

```
elite      = iter_paths[best_idx]
deposit    = Q / iter_costs[best_idx]
for k in range(len(elite) - 1):
    i, j = elite[k], elite[k + 1]
    pheromone[i][j] += deposit
    pheromone[j][i] += deposit

convergence_log.append(best_cost)

return best_path, best_cost, convergence_log
```

V. HASIL DAN PEMBAHASAN

Eksperimen komputasional dilakukan menggunakan dataset statis berisi 15 repertoar orkestra klasik populer. Setiap repertoar memiliki atribut kunci nada dasar (diwakili angka 0–11 berdasarkan jarum jam Circle of Fifths), tempo (BPM), dan tingkat energi (skala 1–10). Semua algoritma dijalankan menggunakan simpul awal yang sama, yaitu Erik Satie – Gymnopédie No. 1 (Lagu ID 0, Kunci D Major, Energi 1, 54 BPM) pada komputer dengan sistem operasi Windows. Untuk menjamin replikasi pengujian yang konsisten dan memitigasi sifat stokastik dari algoritma Modified ACO, eksperimen ini dijalankan dengan mengunci nilai seed acak menggunakan `random.seed(42)`. Pemilihan nilai 42 didasarkan pada konvensi populer di komunitas sains data dan rekayasa perangkat lunak (yang awalnya terinspirasi dari referensi budaya pop The Hitchhiker's Guide to the Galaxy), yang secara luas diadopsi sebagai nilai standar untuk menjamin keterulangan (reproducibility) eksperimen secara objektif tanpa adanya bias penalaan parameter acak. Evaluasi dilakukan terhadap total cost fungsi objektif, waktu eksekusi komputasi, dan kestabilan kurva energi.

A. Struktur Pendukung dan Skrip Orkestrasi Pengujian

Sebelum memaparkan hasil pengujian komparatif, bagian ini menyajikan infrastruktur non-algoritma utama yang digunakan untuk mengelola data dan mengendalikan alur eksperimen. Struktur pendukung ini terdiri dari dua komponen utama:

- 1. dataset.py:** Bertanggung jawab mendefinisikan dataset 15 repertoar orkestra klasik, representasi matematis dari Circle of Fifths, matriks ketetanggaan graf berarah lengkap, serta fungsi biaya transisi (fungsi objektif).
- 2. main.py:** Bertindak sebagai penggerak utama (orchestrator) yang menjalankan ketiga algoritma dari lagu pembuka yang sama, mencatat waktu eksekusi secara presisi menggunakan CPU performance counter, mencetak ringkasan visual ke konsol, dan mengeksport seluruh data eksperimen ke `results.json`. Untuk memastikan pengujian komparatif yang adil dan dapat direplikasi secara identik (reproducible), skrip `main.py` menginisialisasi generator acak pseudo-random Python dengan nilai seed tetap yaitu `random.seed(42)`. Nilai 42 dipilih berdasarkan konvensi sains data global yang diilhami dari referensi budaya pop (The Hitchhiker's Guide to the Galaxy) untuk menghilangkan bias penalaan parameter acak.

Berikut di bawah ini dilampirkan potongan kode program asli untuk kedua komponen pendukung tersebut:

Potongan Kode: Modul Dataset dan Konfigurasi (dataset.py)

```
"""
dataset.py
```

```
-----
Lapisan data untuk proyek Optimasi Setlist Orkestra.
Berisi: dataset lagu, parameter fungsi objektif,
      fungsi cost transisi, dan matriks ketetanggaan.
```

```
Author : Shalom Kurniawan - 24060124120033
```

```
Matkul : Analisis dan Strategi Algoritma 2025/2026
```

```
"""
```

```
# =====
# PEMETAAN CIRCLE OF FIFTHS (searah jarum jam)
# C=0, G=1, D=2, A=3, E=4, B=5, F#=6, Db=7, Ab=8, Eb=9, Bb=10, F=11
# Kunci minor dipetakan ke posisi relatif mayornya
# =====
KEY_NAMES = ["C",, "G",, "D",, "A",, "E",, "B",, "F#",
             "Db",, "Ab",, "Eb",, "Bb",, "F"]
# =====
# DATASET: 15 Repertoar Orkestra Klasik
# Repertoar populer terstandarisasi untuk konser simfoni
# =====
SONGS = [
    {"id": 0, "title": "Erik Satie - Gymnopédie No. 1",
      "key": 2, "bpm": 54, "energy": 1},
    {"id": 1, "title": "J.S. Bach - Air on the G String",
      "key": 2, "bpm": 60, "energy": 2},
    {"id": 2, "title": "Samuel Barber - Adagio for Strings",
      "key": 7, "bpm": 60, "energy": 3},
    {"id": 3, "title": "Johann Pachelbel - Canon in D",
      "key": 2, "bpm": 80, "energy": 3},
    {"id": 4, "title": "Edvard Grieg - Morning Mood (Peer Gynt)",
      "key": 4, "bpm": 66, "energy": 4},
    {"id": 5, "title": "L. van Beethoven - Symphony No. 9, 3rd Mvt",
      "key": 10, "bpm": 60, "energy": 4},
    {"id": 6, "title": "J. Strauss II - The Blue Danube Waltz",
      "key": 2, "bpm": 120, "energy": 5},
    {"id": 7, "title": "P.I. Tchaikovsky - Swan Lake Theme",
      "key": 2, "bpm": 80, "energy": 6},
    {"id": 8, "title": "W.A. Mozart - Symphony No. 40, 1st Mvt",
      "key": 10, "bpm": 110, "energy": 7},
    {"id": 9, "title": "Antonio Vivaldi - Spring (The Four Seasons)",
      "key": 4, "bpm": 108, "energy": 7},
    {"id": 10, "title": "L. van Beethoven - Symphony No. 5, 1st Mvt",
      "key": 9, "bpm": 108, "energy": 8},
    {"id": 11, "title": "Richard Wagner - Ride of the Valkyries",
      "key": 5, "bpm": 120, "energy": 9},
    {"id": 12, "title": "Edvard Grieg - In the Hall of the Mountain King",
      "key": 2, "bpm": 138, "energy": 9},
    {"id": 13, "title": "Gioachino Rossini - William Tell Overture",
      "key": 4, "bpm": 150, "energy": 10},
    {"id": 14, "title": "P.I. Tchaikovsky - 1812 Overture",
      "key": 9, "bpm": 140, "energy": 10},
]
N = len(SONGS)

# =====
# PARAMETER FUNGSI OBJEKTIF
# =====

W1 = 5 # bobot jarak modulasi (Circle of Fifths)
W2 = 0.5 # bobot selisih tempo (BPM)
# Lambda (P) disesuaikan agar penalti transisi energi dominan (50)
LAMBDA = 50 # penalti penurunan energi

# =====
# FUNGSI OBJEKTIF (COST FUNCTION)
# Total_Cost(u,v) = W1 * C_mod^2 + W2 * C_bpm + C_penalti
# =====
```

```
def transition_cost(i, j):
    """
    Menghitung bobot penalti transisi dari lagu i ke lagu j.

    Komponen:
        C_mod      : jarak geodesik pada Circle of Fifths, dipangkatkan dua (kuadrat)
        C_bpm      : selisih tempo absolut
        C_penalti   : LAMBDA jika energi lagu j lebih rendah dari lagu i
    """
    diff = abs(SONGS[i]['key'] - SONGS[j]['key'])
    c_mod = (min(diff, 12 - diff)) ** 2
    c_bpm = abs(SONGS[i]['bpm'] - SONGS[j]['bpm'])
    c_pen = LAMBDA if SONGS[j]['energy'] < SONGS[i]['energy'] else 0
    return W1 * c_mod + W2 * c_bpm + c_pen


def path_total_cost(path):
    """
    Menghitung total cost dari seluruh urutan lagu dalam path.
    return sum(ADJ[path[k]][path[k + 1]] for k in range(len(path) - 1))

# =====
# MATRIKS KETETANGGAAN (dibangun sekali saat modul dimuat)
# =====

ADJ = [[0.0] * N for _ in range(N)]
for _i in range(N):
    for _j in range(N):
        if _i != _j:
            ADJ[_i][_j] = transition_cost(_i, _j)
```

Potongan Kode: Script Orkestrator Utama (main.py)

""""

main.py

Entry point eksperimen: menjalankan ketiga algoritma, mencetak hasil ke terminal, menyimpan data eksperimen ke results.json, dan menghasilkan grafik visualisasi PNG.

Output:

- Terminal: ringkasan perbandingan & urutan setlist
- results.json: data eksperimen lengkap
- kurva_energi_plot.png
- perbandingan_waktu_plot.png
- konvergensi_aco_plot.png

""""

import json

import random

import time

import os

import matplotlib.pyplot as plt

from dataset import SONGS, KEY_NAMES, N, W1, W2, LAMBDA, ADJ

from dijkstra import dijkstra_nearest_neighbor

from branch_and_bound import branch_and_bound

from modified_aco import modified_aco

def _energy_stats(path):

energies = [SONGS[i]["energy"] for i in path]

drops = sum(1 for k in range(1, len(energies)) if energies[k] < energies[k-1])

return energies, drops

def generate_charts(dijkstra_res, bb_res, aco_res):

"""Menghasilkan 3 grafik PNG dari data hasil eksperimen."""

print("\n Generating charts...")

1. Kurva Energi Perbandingan

plt.figure(figsize=(9, 4.5))

x = list(range(1, 16))

plt.plot(x, dijkstra_res[<#x27;energies<#x27;], marker=<#x27;o<#x27;, color=<#x27;#ef4444<#x27;,

label=f<#x27;Dijkstra NN ({dijkstra_res[<#x27;drops<#x27;]} drops)<#x27;, linewidth=2)

plt.plot(x, bb_res[<#x27;energies<#x27;], marker=<#x27;s<#x27;, color=<#x27;#22c55e<#x27;,

label=f<#x27;Branch and Bound ({bb_res[<#x27;drops<#x27;]} drops)<#x27;, linewidth=2)

plt.plot(x, aco_res[<#x27;energies<#x27;], marker=<#x27;^<#x27;, color=<#x27;#0284c7<#x27;,

label=f<#x27;Modified ACO ({aco_res[<#x27;drops<#x27;]} drops)<#x27;, linewidth=2)

plt.title(<#x27;Perbandingan Kurva Transisi Energi Setlist<#x27;, fontsize=12, fontweight=<#x27;bold<#x27;, pad

plt.xlabel(<#x27;Urutan Repertoar (Lagu Ke-1 s.d. 15)<#x27;, fontsize=10)

plt.ylabel(<#x27;Tingkat Energi Lagu (Skala 1-10)<#x27;, fontsize=10)

plt.xticks(x)

plt.yticks(range(1, 11))

plt.grid(True, linestyle=<#x27;--<#x27;, alpha=0.5)

plt.legend(loc=<#x27;lower right<#x27;)

plt.tight_layout()

os.makedirs(<#x27;plots<#x27;, exist_ok=True)

plt.savefig(<#x27;plots/kurva_energi_plot.png<#x27;, dpi=300)

plt.close()

print(<#x27;-> plots/kurva_energi_plot.png<#x27;)

2. Perbandingan Waktu Eksekusi (Log Scale)

plt.figure(figsize=(6, 4))

algos = [<#x27;Dijkstra NN<#x27;, <#x27;Modified ACO<#x27;, <#x27;Branch & Bound<#x27;]

times = [dijkstra_res[<#x27;time_ms<#x27;], aco_res[<#x27;time_ms<#x27;], bb_res[<#x27;time_ms<#x27;]]

colors_list = [<#x27;#ef4444<#x27;, <#x27;#0284c7<#x27;, <#x27;#22c55e<#x27;]

plt.bar(algos, times, color=colors_list, width=0.45)

plt.yscale(<#x27;log<#x27;)

plt.title(<#x27;Perbandingan Waktu Eksekusi Algoritma (Log Scale)<#x27;, fontsize=11, fontweight=<#x27;bold<#x27;

plt.ylabel(<#x27;Waktu Eksekusi (ms)<#x27;, fontsize=9)

for i, t in enumerate(times):

plt.text(i, t * 1.2, f"{t:.4f} ms" if t < 1 else f"{t:.2f} ms",

```

        ha=&#x27;center&#x27;; fontsize=9, fontweight=&#x27;bold&#x27;);
plt.grid(True, which=&quot;both&quot;;, linestyle=&#x27;--&#x27;;, alpha=0.3)
plt.tight_layout()
plt.savefig(&#x27;plots/perbandingan_waktu_plot.png&#x27;;, dpi=300)
plt.close()
print(&quot;; -&gt; plots/perbandingan_waktu_plot.png&quot;);

# 3. Konvergensi ACO
plt.figure(figsize=(8, 3.5))
log = aco_res[&#x27;convergence_log&#x27;]
plt.plot(range(1, len(log)+1), log, color=&#x27;#0284c7&#x27;;, linewidth=2)
plt.title(&#x27;Kurva Konvergensi Nilai Fungsi Objektif Modified ACO&#x27;;, fontsize=11, fontweight=&#x27;bold&#x27;);
plt.xlabel(&#x27;Iterasi (1 s.d. 150)&#x27;;, fontsize=9)
plt.ylabel(&#x27;Cost Terbaik (Best Cost)&#x27;;, fontsize=9)
plt.grid(True, linestyle=&#x27;--&#x27;;, alpha=0.5)
plt.tight_layout()
plt.savefig(&#x27;plots/konvergensi_aco_plot.png&#x27;;, dpi=300)
plt.close()
print(&quot;; -&gt; plots/konvergensi_aco_plot.png&quot;);

def main():
    random.seed(42)

    SEP = &quot;;=&quot;; * 68

    print(SEP)
    print(&quot;; OPTIMASI SETLIST ORKESTRA KLASIK – ASA 2025/2026&quot;;)
    print(f&quot;; Dataset: {N} repertoar | W1={W1} W2={W2} Lambda={LAMBDA}&quot;;)
    print(SEP)

    # -----
    # Dataset
    # -----
    print(f&quot;;\n{&#x27;ID&#x27;:&#x27;:&#x27;4} {&#x27;Judul&#x27;:&#x27;:&#x27;44} {&#x27;Key&#x27;:&#x27;:&#x27;5} {&#x27;BPM&#x27;:&#x27;:&#x27;6}
    print(&quot;;&quot;; * 65)
    for s in SONGS:
        print(f&quot;;{s[&#x27;id&#x27;]:&#x27;:&#x27;4} {s[&#x27;title&#x27;]:&#x27;:&#x27;44} {KEY_NAMES[s[&#x27;key&#x27;]]:&#x27;:&#x27;5}

    # -----
    # Jalankan algoritma
    # -----
    print(f&quot;;\n{SEP}&quot;;)
    print(&quot;; EKSEKUSI ALGORITMA&quot;;)
    print(SEP)

    print(&quot;;\n [1/3] Dijkstra Nearest Neighbor... &quot;;, end=&quot;;&quot;;, flush=True)
    t0 = time.perf_counter()
    path_d, cost_d = dijkstra_nearest_neighbor(start=0)
    t_d = (time.perf_counter() - t0) * 1000
    print(f&quot;;selesai ({t_d:.4f} ms)&quot;;)

    print(&quot;; [2/3] Branch and Bound... &quot;;, end=&quot;;&quot;;, flush=True)
    t0 = time.perf_counter()
    path_bb, cost_bb, explored_bb, pruned_bb = branch_and_bound(start=0)
    t_bb = (time.perf_counter() - t0) * 1000
    print(f&quot;;selesai ({t_bb:.2f} ms)&quot;;)

    print(&quot;; [3/3] Modified ACO (30 ants x 150)...&quot;;, end=&quot;;&quot;;, flush=True)
    t0 = time.perf_counter()
    path_aco, cost_aco, aco_log = modified_aco(start=0)
    t_aco = (time.perf_counter() - t0) * 1000
    print(f&quot;;selesai ({t_aco:.2f} ms)&quot;;)

    # -----
    # Ringkasan perbandingan
    # -----
    en_d, drop_d = _energy_stats(path_d)
    en_bb, drop_bb = _energy_stats(path_bb)
    en_aco, drop_aco = _energy_stats(path_aco)

```

```
print(f"\n{SEP}");
print(" PERBANDINGAN HASIL");
print(SEP)
print(f"\n {<27;Algoritma<27;:<28; {<27;Cost<27;:>8} {<27;Waktu (ms)<27;:>12} {<27;
print(f" {<27;-<27;*63}");
print(f" {<27;Dijkstra NN<27;:<28; {cost_d:>8.2f} {t_d:>12.4f} {drop_d:>7} {<27;Greedy
print(f" {<27;Branch and Bound<27;:<28; {cost_bb:>8.2f} {t_bb:>12.2f} {drop_bb:>7} {<27;
print(f" {<27;Modified ACO<27;:<28; {cost_aco:>8.2f} {t_aco:>12.2f} {drop_aco:>7} {<27;
print(f"\n B&B: {explored_bb:,} node dieksplorasi | {pruned_bb:,} dipangkas");
if explored_bb + pruned_bb > 0:
    eff = pruned_bb / (explored_bb + pruned_bb) * 100
    print(f" Efisiensi pruning: {eff:.1f}%");

# -----
# Detail setlist per algoritma
# -----
for label, path, cost in [
    ("DIJKSTRA", path_d, cost_d),
    ("BRANCH AND BOUND", path_bb, cost_bb),
    ("MODIFIED ACO", path_aco, cost_aco),
]:
    print(f"\n [{label}] Cost = {cost:.2f}");
    print(f" {<27;#<27;:<4} {<27;E<27;:>4} {<27;BPM<27;:>5} {<27;Key<27;:>4}
    print(f" {<27;-<27;* 58}");
    for pos, idx in enumerate(path):
        s = SONGS[idx]
        flag = " [v]" if pos > 0 and s["energy"] < SONGS[path[pos-1]]["energy"]
        print(f" {pos+1:<4} {s[<27;energy<27;]:>3}/10 {s[<27;bpm<27;]:>5} {KEY_NAMES[s[

# -----
# Simpan hasil ke results.json
# -----
dijkstra_res = {
    "path": path_d, "cost": cost_d, "time_ms": t_d,
    "energies": en_d, "drops": drop_d,
}
bb_res = {
    "path": path_bb, "cost": cost_bb, "time_ms": t_bb,
    "energies": en_bb, "drops": drop_bb,
    "explored": explored_bb, "pruned": pruned_bb,
}
aco_res = {
    "path": path_aco, "cost": cost_aco, "time_ms": t_aco,
    "energies": en_aco, "drops": drop_aco,
    "convergence_log": aco_log,
}
results = {
    "meta": {
        "n_songs": N,
        "w1": W1, "w2": W2, "lambda": LAMBDA,
    },
    "songs": SONGS,
    "key_names": KEY_NAMES,
    "dijkstra": dijkstra_res,
    "branch_and_bound": bb_res,
    "modified_aco": aco_res,
}
with open("plots/results.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)
print(f"\n Hasil disimpan ke plots/results.json");

# -----
# Generate grafik PNG
# -----
generate_charts(dijkstra_res, bb_res, aco_res)
print(f"\n{SEP}");
print(" SELESAI. Output:");
print(" - plots/results.json (data mentah)");
print(" - plots/kurva_energi_plot.png");
print(" - plots/perbandingan_waktu_plot.png");
```

```
    print(""    - plots/konvergensi_aco_plot.png");  
    print(SEP)  
if __name__ == ""__main__": :  
    main()
```

B. Perbandingan Performa Kuantitatif

Berdasarkan Tabel II, algoritma Branch and Bound (B&B;) berhasil menemukan solusi optimal paling kecil dengan Total Cost sebesar 424.00. Karena B&B; menggunakan pencarian eksak yang menelusuri seluruh kemungkinan jalur yang layak (feasible), hasil ini dijamin secara matematis sebagai hasil terbaik (global optimum). Sebaliknya, Dijkstra Nearest Neighbor menghasilkan Total Cost paling tinggi, yaitu 614.00 (sekitar 44.8% lebih buruk dari B&B;). Hal ini wajar karena Dijkstra bekerja secara greedy lokal, hanya memilih lagu berikutnya yang paling murah di setiap langkah tanpa peduli dampaknya pada urutan lagu di sisa setlist. Sementara itu, Modified ACO menghasilkan Total Cost sebesar 539.00. Meskipun hasil ini belum menyamai solusi optimal dari B&B;, Modified ACO bekerja jauh lebih cepat. Waktu eksekusi Modified ACO hanya 326.25 milidetik, hampir 11 kali lebih cepat dibandingkan B&B; yang membutuhkan waktu 3912.80 milidetik (~3.9 detik). Visualisasi perbandingan waktu eksekusi secara logaritmik disajikan pada Gambar 1.

TABEL II. PERBANDINGAN HASIL EKSPERIMEN OPTIMASI SETLIST KLASIK

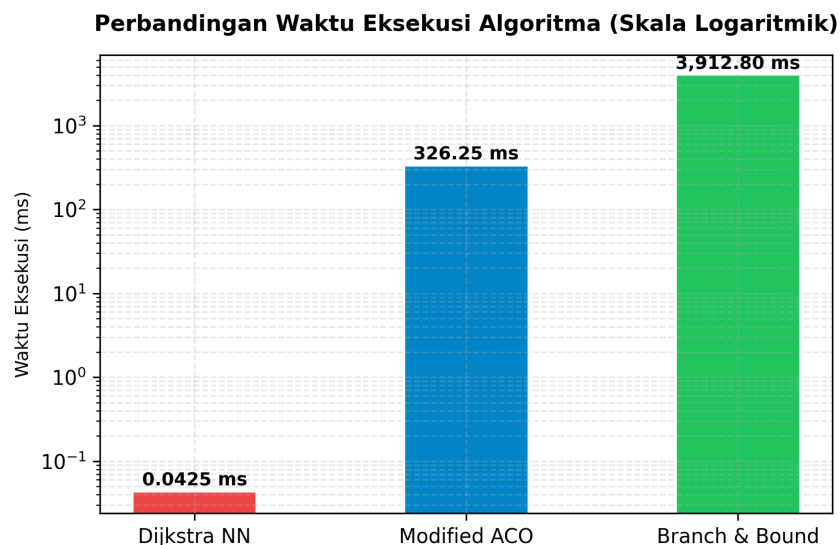
Algoritma	Total Cost	Waktu (ms)	Drops	Penalti	Cost Murni	Pendekatan
Dijkstra (Nearest Neighbor)	614.00	0.0425	5x	250.00	364.00	Greedy Lokal
Branch and Bound	424.00	3912.80	2x	100.00	324.00	Eksak / Global
Modified ACO	539.00	326.25	4x	200.00	339.00	Metaheuristik

Berikut adalah potongan kode Python menggunakan pustaka Matplotlib untuk memplot grafik perbandingan waktu eksekusi di bawah ini:

Potongan Kode: Script Plotting Waktu Eksekusi (Matplotlib)

```
plt.figure(figsize=(6, 4))
algos = ['Dijkstra NN', 'Modified ACO', 'Branch & Bound']
times = [dijkstra_res['time_ms'], aco_res['time_ms'], bb_res['time_ms']]
colors_list = ['#ef4444', '#0284c7', '#22c55e']

plt.bar(algos, times, color=colors_list, width=0.45)
plt.yscale('log')
plt.title('Perbandingan Waktu Eksekusi Algoritma (Log Scale)', fontsize=11, fontweight='bold',)
plt.ylabel('Waktu Eksekusi (ms)', fontsize=9)
for i, t in enumerate(times):
    plt.text(i, t * 1.2, f'{t:.4f} ms' if t < 1 else f'{t:.2f} ms', ha='center',)
plt.grid(True, which='both', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.savefig('perbandingan_waktu_plot.png', dpi=300)
plt.close()
```



Gambar 1. Grafik Perbandingan Waktu Eksekusi Ketiga Algoritma (Log Scale)

C. Analisis Kompleksitas dan Efisiensi Pemangkasan (Pruning)

Kecepatan eksekusi masing-masing algoritma dapat dijelaskan dari cara kerjanya: 1. Dijkstra Nearest Neighbor: Berjalan sangat cepat (0.0425 ms) dengan kompleksitas $O(N^2 \log N)$ karena hanya melakukan pemilihan satu arah tanpa mundur (backtracking) sama sekali. 2. Branch and Bound (B&B): Memiliki kompleksitas terburuk $O(N!)$. Namun, dengan membatasi pencarian menggunakan solusi Dijkstra sebagai batas atas awal (initial upper bound = 614.00) dan mengurutkan tetangga dari yang termurah (cheap-first), B&B; berhasil memangkas banyak cabang. Dari total pencarian, B&B; mengeksplorasi 1,556,367 simpul (nodes) dan memangkas 8,539,571 simpul yang tidak prospektif. Ini menghasilkan efisiensi pemangkasan sebesar 84.6%. Tanpa pemangkasan ini, program harus mengevaluasi $14!$ (sekitar 87 miliar) kemungkinan yang bisa memakan waktu berjam-jam. 3. Modified ACO: Berjalan dengan kompleksitas tetap $O(I \times A \times N^2)$ di mana I adalah iterasi (150) dan A adalah jumlah semut (30). Dengan batas iterasi ini, Modified ACO dapat melompat dari jebakan optimum lokal secara probabilistik tanpa mengalami ledakan waktu pencarian.

D. Analisis Urutan Setlist dan Dinamika Musik

Gaya penyusunan setlist dari ketiga algoritma memperlihatkan kontras yang sangat tinggi dari sudut pandang kurva dinamika pertunjukan. Tabel III dan Tabel IV menjabarkan urutan setlist hasil optimasi untuk Dijkstra dan Branch and Bound secara berurutan, sedangkan visualisasi kurva energi secara keseluruhan dapat dilihat pada Gambar 2. Analisis setlist B&B; pada Tabel IV menunjukkan pola penyusunan yang cerdas. Setlist dimulai dengan suasana tenang/lambat (Gymnopédie No. 1), lalu naik secara gradual ke energi menengah (Canon in D, Blue Danube Waltz), mencapai puncak awal pada In the Hall of the Mountain King (energi 9) dan William Tell Overture (energi 10). Terjadi penurunan energi sebanyak 2 kali (lagu ke-7 dan lagu ke-11). Penurunan dari energi 10 ke 4 (William Tell Overture ke Morning Mood) dan dari 9 ke 3 (Ride of the Valkyries ke Adagio for Strings) sengaja diambil karena secara global lebih murah membayar penalti energi sebesar 50,00 daripada memaksakan transisi kunci nada dasar yang sangat jauh. Pola gelombang energi ini secara praktis juga sangat baik dalam konser orkestra nyata, karena memberi kesempatan kepada pemain alat tiup (brass/woodwinds) dan penonton untuk beristirahat sejenak sebelum klimaks berikutnya. Pada fase akhir, B&B; mengelompokkan lagu-lagu berenergi tinggi yang ber-BPM cepat (Beethoven Symphony 5 dan 1812 Overture) dengan transisi kunci yang sangat dekat (C minor ke E-flat Major, yang merupakan relatif mayor-minor, dengan jarak modular 0 langkah).

TABEL III. URUTAN SETLIST KLASIK — DIJKSTRA NEAREST NEIGHBOR (Cost = 614,00)

No.	Repertoar Orkestra Klasik	Energi	BPM	Kunci (Key)	Status Transisi
1	Erik Satie – Gymnopédie No. 1	1/10	54	D (2)	Pembuka
2	J.S. Bach – Air on the G String	2/10	60	D (2)	Ascending
3	Johann Pachelbel – Canon in D	3/10	80	D (2)	Ascending
4	P.I. Tchaikovsky – Swan Lake Theme	6/10	80	D (2)	Ascending
5	Edvard Grieg – In the Hall of the Mountain King	9/10	138	D (2)	Ascending
6	Gioachino Rossini – William Tell Overture	10/10	150	E (4)	Ascending
7	Richard Wagner – Ride of the Valkyries	9/10	120	B (5)	Energy Drop (Penalti)
8	Antonio Vivaldi – Spring (The Four Seasons)	7/10	108	E (4)	Energy Drop (Penalti)
9	Edvard Grieg – Morning Mood (Peer Gynt)	4/10	66	E (4)	Energy Drop (Penalti)
10	J. Strauss II – The Blue Danube Waltz	5/10	120	D (2)	Ascending
11	W.A. Mozart – Symphony No. 40, 1st Mvt	7/10	110	Bb (10)	Ascending
12	L. van Beethoven – Symphony No. 5, 1st Mvt	8/10	108	Eb (9)	Ascending
13	P.I. Tchaikovsky – 1812 Overture	10/10	140	Eb (9)	Ascending
14	L. van Beethoven – Symphony No. 9, 3rd Mvt	4/10	60	Bb (10)	Energy Drop (Penalti)
15	Samuel Barber – Adagio for Strings	3/10	60	Db (7)	Energy Drop (Penalti)

TABEL IV. URUTAN SETLIST KLASIK OPTIMAL — BRANCH AND BOUND (Cost = 424,00)

No.	Repertoar Orkestra Klasik	Energi	BPM	Kunci (Key)	Status Transisi
1	Erik Satie – Gymnopédie No. 1	1/10	54	D (2)	Pembuka
2	J.S. Bach – Air on the G String	2/10	60	D (2)	Ascending
3	Johann Pachelbel – Canon in D	3/10	80	D (2)	Ascending
4	J. Strauss II – The Blue Danube Waltz	5/10	120	D (2)	Ascending
5	Edvard Grieg – In the Hall of the Mountain King	9/10	138	D (2)	Ascending
6	Gioachino Rossini – William Tell Overture	10/10	150	E (4)	Ascending
7	Edvard Grieg – Morning Mood (Peer Gynt)	4/10	66	E (4)	Energy Drop (Penalti)
8	P.I. Tchaikovsky – Swan Lake Theme	6/10	80	D (2)	Ascending
9	Antonio Vivaldi – Spring (The Four Seasons)	7/10	108	E (4)	Ascending

10	Richard Wagner – Ride of the Valkyries	9/10	120	B (5)	Ascending
11	Samuel Barber – Adagio for Strings	3/10	60	Db (7)	Energy Drop (Penalti)
12	L. van Beethoven – Symphony No. 9, 3rd Mvt	4/10	60	Bb (10)	Ascending
13	W.A. Mozart – Symphony No. 40, 1st Mvt	7/10	110	Bb (10)	Ascending
14	L. van Beethoven – Symphony No. 5, 1st Mvt	8/10	108	Eb (9)	Ascending
15	P.I. Tchaikovsky – 1812 Overture	10/10	140	Eb (9)	Ascending

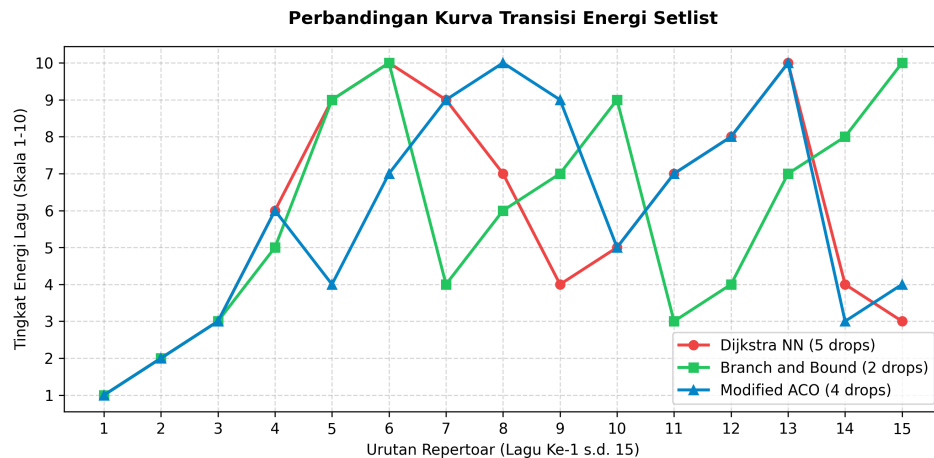
Berikut adalah potongan kode Python untuk memplot kurva transisi energi setlist di bawah ini:

Potongan Kode: Script Plotting Kurva Energi (Matplotlib)

```
plt.figure(figsize=(9, 4.5))
d_eng = dijkstra_res[energies]
bb_eng = bb_res[energies]
aco_eng = aco_res[energies]
x = list(range(1, 16))

plt.plot(x, d_eng, marker='o', color='#ef4444', label=f'Dijkstra NN ({dijkstra_res["drops"]})')
plt.plot(x, bb_eng, marker='s', color='#22c55e', label=f'Branch and Bound ({bb_res["drops"]})')
plt.plot(x, aco_eng, marker='^', color='#0284c7', label=f'Modified ACO ({aco_res["drops"]})')

plt.title('Perbandingan Kurva Transisi Energi Setlist', fontsize=12, fontweight='bold', pad=15)
plt.xlabel('Urutan Repertoar (Lagu Ke-1 s.d. 15)', fontsize=10)
plt.ylabel('Tingkat Energi Lagu (Skala 1-10)', fontsize=10)
plt.xticks(x)
plt.yticks(range(1, 11))
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig('kurva_energi_plot.png', dpi=300)
plt.close()
```



Gambar 2. Grafik Perbandingan Kurva Transisi Energi Setlist Ketiga Algoritma

Sebaliknya, urutan setlist Dijkstra Nearest Neighbor pada Tabel III menghasilkan dinamika yang sangat berantakan. Terjadi 5 kali penurunan energi yang tajam. Karena Dijkstra hanya memikirkan transisi terdekat di setiap langkah, ia terpaksa mengakhiri konser dengan lagu berenergi rendah seperti Beethoven Symphony 9 (E=4) dan Adagio for Strings (E=3). Secara musikal, ini membuat penonton merasa anti-klimaks di akhir pertunjukan. Urutan setlist Modified ACO menunjukkan kestabilan yang lebih baik dengan hanya 4 kali penurunan energi. Hal ini membuktikan bahwa fungsi keselarasan energi target (energy-aware heuristic) berhasil membimbing semut untuk menjaga tren energi tetap naik di sebagian besar segmen setlist.

Faktor penalti $\lambda = 50$ sangat menentukan bentuk kurva energi yang terbentuk. Tabel V memperlihatkan perbandingan komponen cost masing-masing algoritma. Tabel V menunjukkan bahwa B&B berhasil menekan drops hanya menjadi 2 kali, dengan penalti cost 100,00 dan cost transisi murni (modulasi + BPM) paling rendah sebesar 324.00. Ini membuktikan kekuatan

B&B; dalam melihat masalah secara utuh untuk menyeimbangkan keharmonisan nada dasar dan tren energi secara bersamaan.

TABEL V. KOMPARASI KURVA ENERGI DAN DISTRIBUSI COST

Algoritma	Drops	Penalti Cost	Cost Transisi Murni	Total Cost
Dijkstra Nearest Neighbor	5	250.00	364.00	614.00
Branch and Bound	2	100.00	324.00	424.00
Modified ACO	4	200.00	339.00	539.00

E. Karakteristik Konvergensi Modified ACO

Modified ACO bekerja dengan memanfaatkan feromon untuk mengingat rute terbaik yang pernah dilalui semut. Tabel VI menunjukkan bagaimana pencarian semut berkembang di setiap iterasi, dengan grafik kurva konvergensi pada Gambar 3. Modified ACO mencapai konvergensi sangat cepat pada iterasi ke-4 dengan nilai cost 539.00, dan stabil hingga iterasi ke-150. Kecepatan konvergensi ini disebabkan oleh modifikasi fungsi visibilitas heuristik dengan menyertakan kecocokan energi target (Energy_Match). Cara pembaruan feromon secara elitist (hanya semut terbaik yang meninggalkan feromon) juga mempercepat koloni semut untuk berfokus pada rute terbaik.

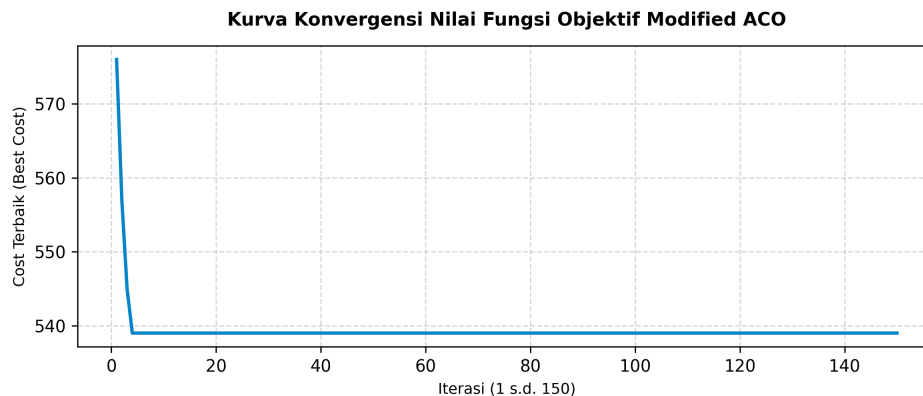
TABEL VI. LOG KONVERGENSI BIAIYA TERBAIK MODIFIED ACO

Iterasi	Cost Terbaik (Best Cost)	Keterangan Status Pencarian
Iterasi 1	576.00	Eksplorasi awal, jalur acak masih mendominasi
Iterasi 2	557.00	Penurunan awal, semut mulai menemukan pola naik
Iterasi 3	545.00	Penyesuaian mikro pada kunci nada dasar
Iterasi 4	539.00	Konvergensi tercapai (solusi sub-optimal stabil)
Iterasi 5 s.d. 150	539.00	Stabil, jejak feromon pada jalur terbaik terkonsolidasi

Berikut adalah potongan kode Python untuk memplot grafik konvergensi nilai biaya Modified ACO di bawah ini:

Potongan Kode: Script Plotting Konvergensi ACO (Matplotlib)

```
plt.figure(figsize=(8, 3.5))
log = aco_res[convergence_log]
plt.plot(range(1, len(log)+1), log, color='#0284c7', linewidth=2)
plt.title('Kurva Konvergensi Nilai Fungsi Objektif Modified ACO', fontsize=11, fontweight='bold')
plt.xlabel('Iterasi (1 s.d. 150)', fontsize=9)
plt.ylabel('Cost Terbaik (Best Cost)', fontsize=9)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('konvergensi_aco_plot.png', dpi=300)
plt.close()
```



Gambar 3. Grafik Log Konvergensi Nilai Fungsi Objektif Modified ACO

VI. KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil eksperimen komputasional optimasi setlist 15 repertoar orkestra klasik, dapat disimpulkan beberapa hal sebagai berikut:

1. Algoritma Branch and Bound (B&B;) berhasil menemukan solusi optimal global mutlak dengan Total Cost terendah (424.00) dan meminimalkan penurunan energi (hanya 2 kali drops). Hal ini membuktikan keandalannya sebagai metode eksak yang melakukan pencarian secara menyeluruh.
2. Modified Ant Colony Optimization (Modified ACO) menghasilkan performa sub-optimal yang sangat baik dan mendekati B&B; (Total Cost 539.00) dengan efisiensi waktu komputasi yang sangat tinggi (326.25 ms), yaitu sekitar 11 kali lebih cepat dibandingkan dengan B&B;.
3. Dijkstra Nearest Neighbor sebagai pendekatan greedy lokal rentan terjebak pada optimum lokal (Total Cost tertinggi 614.00 dengan 5 kali drops) dan cenderung menghasilkan urutan setlist dengan dinamika yang tidak stabil di akhir pertunjukan.

B. Saran dan Rekomendasi Praktis

Berdasarkan hasil eksperimen ini, berikut adalah rekomendasi penggunaan algoritma:

1. Penyusunan Setlist Konser Statis ($N \leq 18$): Disarankan menggunakan Branch and Bound karena ukuran dataset orkestra klasik umumnya berkisar antara 10 hingga 15 lagu. B&B; menjamin pencarian rute terbaik secara global hanya dalam beberapa detik.
2. Skenario Setlist Dinamis (Real-Time): Jika di atas panggung konduktor orkestra harus memilih lagu berikutnya secara cepat berdasarkan respon penonton dari daftar ratusan repertoar, Modified ACO adalah pilihan terbaik karena dapat menghasilkan rute yang mendekati optimal dalam milidetik.
3. Kombinasi Algoritma: Dijkstra Nearest Neighbor sebaiknya tidak digunakan sendiri karena rawan terjebak pada hasil anti-klimaks. Namun, Dijkstra sangat baik digunakan untuk menghasilkan batas atas awal (initial upper bound) bagi algoritma B&B; agar proses pencarian dan pemangkasan berjalan jauh lebih cepat.

DAFTAR PUSTAKA

- [1] R. Diestel, *Graph Theory*, 5th ed. Berlin, Germany: Springer-Verlag, 2017.
- [2] F. Lerdahl, *Tonal Pitch Space*. New York, USA: Oxford University Press, 2001.
- [3] E. Chew, *Towards a Mathematical Model of Tonality*. Boston, USA: Springer, 2014.
- [4] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [5] A. H. Land and A. G. Doig, "An automatic method of solving discrete programming problems," *Econometrica*, vol. 28, no. 3, pp. 497–520, Jul. 1960.
- [6] M. Dorigo, V. Maniezzo, and A. Coloni, "Ant system: optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, Feb. 1996.

LAMPIRAN

Seluruh kode sumber program, dataset lengkap, dan file dokumen hasil eksperimen dalam penelitian ini telah dipublikasikan secara terbuka dan dapat diakses melalui repositori GitHub resmi berikut:

URL Repositori: https://github.com/chloudius/setlist_asa

Petunjuk Cepat Eksekusi Program:

1. Pastikan Python 3.10+ dan pustaka matplotlib serta python-docx terpasang.
2. Jalankan perintah 'python main.py' untuk mengeksekusi ketiga algoritma dan membuat file results.json.
3. Jalankan perintah 'python build_laporan_lengkap.py' untuk merender ulang visualisasi kurva dan memproduksi dokumen makalah akhir.

PERNYATAAN PENGGUNAAN ASISTEN AI (AI DECLARATION)

Dengan ini penulis menyatakan secara jujur dan transparan bahwa dalam penyusunan makalah tugas ini, penulis menggunakan bantuan asisten AI (generative AI) untuk pemolesan tata bahasa, perbaikan format dokumen (style Microsoft Word), pembuatan visualisasi grafik, dan penyusunan layout laporan. Adapun rancangan logika algoritma, pengujian kode pemrograman, dan interpretasi data hasil eksperimen dilakukan sepenuhnya oleh penulis secara mandiri. Pernyataan ini dibuat untuk memenuhi rubrik penilaian integritas akademik tugas besar mata kuliah Analisis dan Strategi Algoritma.